

```
import pandas as pd
df=pd.read_csv('diabetes.csv')
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
from decorator import DEF
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

X=df.drop('Outcome',axis=1)
Y=df['Outcome']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=40)

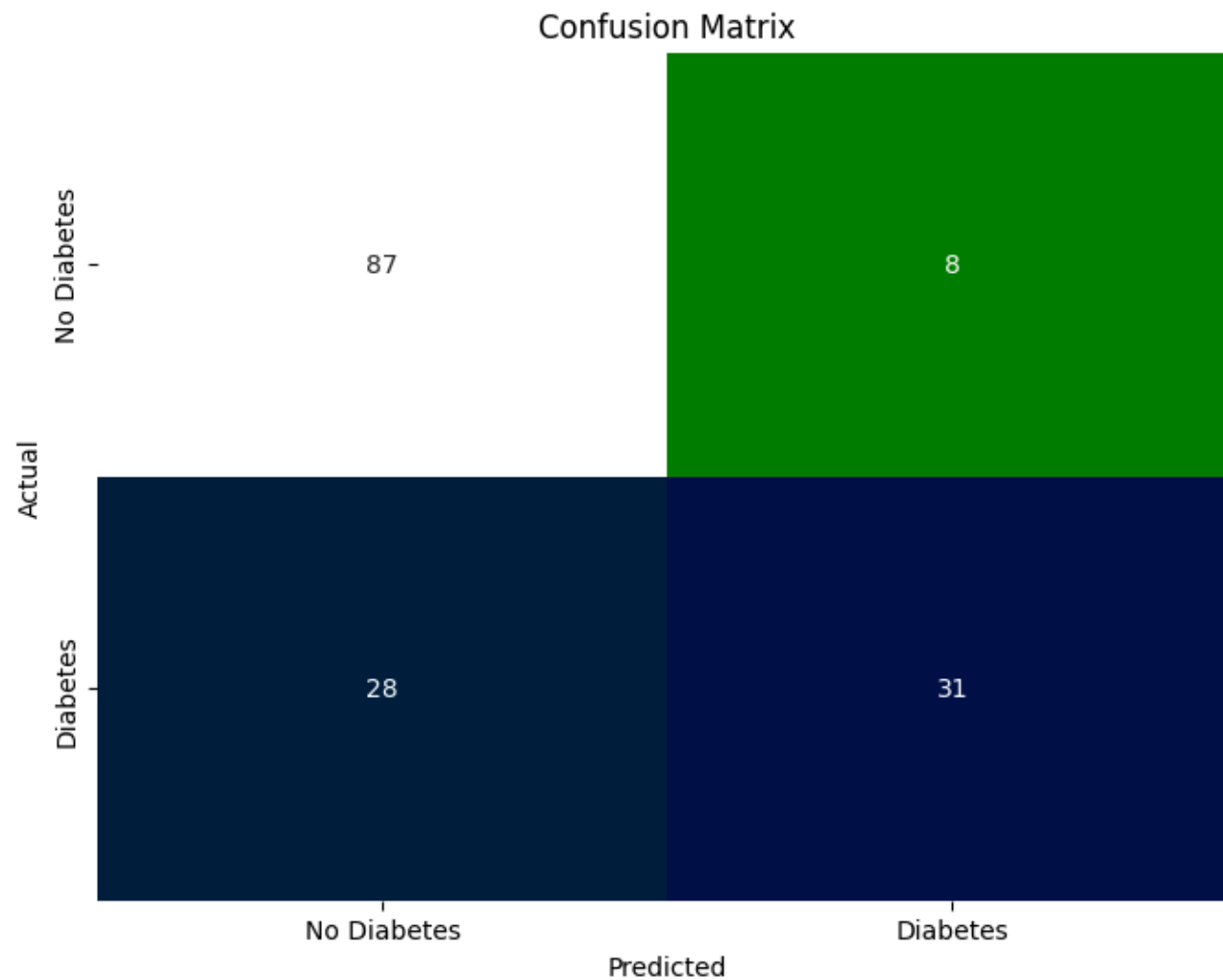
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)
```

```
svm_model = SVC(kernel='linear', C=1)
svm_model.fit(X_train_scaled, y_train)
y_pred=svm_model.predict(X_test_scaled)
```

```
SVMLinear_accuracy = accuracy_score(y_test, y_pred)
precision=precision_score(y_test,y_pred)
recall=recall_score(y_test,y_pred)
f1=f1_score(y_test,y_pred)
conf_matrix=confusion_matrix(y_test,y_pred)
print(f'Accuracy: {SVMLinear_accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print(f'F1 Score: {f1:.4f}')
print('Confusion Matrix:')
print(conf_matrix)
```

```
Accuracy: 0.7662
Precision: 0.7949
Recall: 0.5254
F1 Score: 0.6327
Confusion Matrix:
[[87  8]
 [28 31]]
```

```
plt.figure(figsize=(4, 3))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="ocean", cbar=False, xticklabels=['No Diabetes', 'Diabetes'], yticklabels=
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



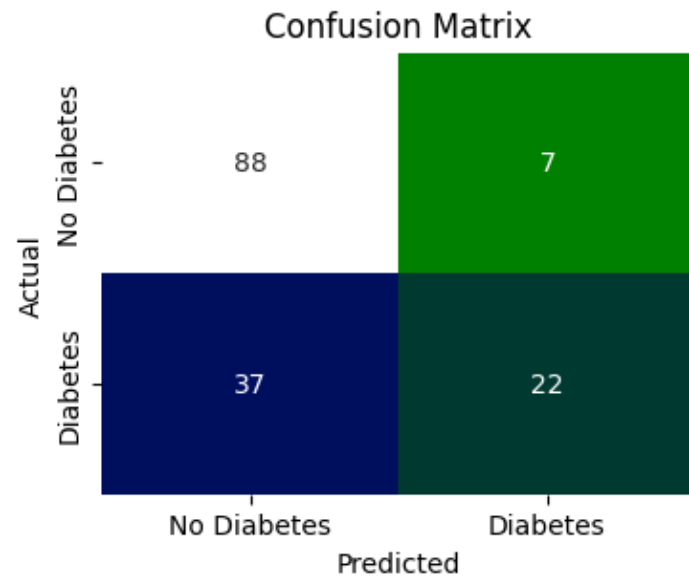
```
svm_model=SVC(kernel='poly',degree=3,C=1)          #c is attribute use for regularization
svm_model.fit(X_train_scaled,y_train)
y_pred=svm_model.predict(X_test_scaled)
```

```
accuracy=accuracy_score(y_test, y_pred)
precision=precision_score(y_test,y_pred)
recall=recall_score(y_test,y_pred)
f1=f1_score(y_test,y_pred)
```

```
conf_matrix=confusion_matrix(y_test,y_pred)
print(f'Accuracy: {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print(f'F1 Score: {f1:.4f}')
print('Confusion Matrix:')
print(conf_matrix)
```

```
Accuracy: 0.7143
Precision: 0.7586
Recall: 0.3729
F1 Score: 0.5000
Confusion Matrix:
[[88  7]
 [37 22]]
```

```
plt.figure(figsize=(4, 3))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="ocean", cbar=False, xticklabels=['No Diabetes', 'Diabetes'], yticklabels=
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
```

```
X=df.drop('Outcome',axis=1)
y=df['Outcome']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

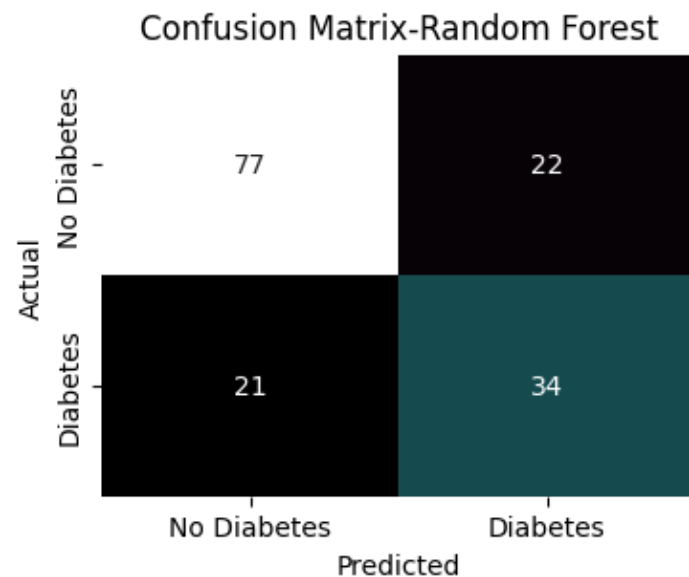
```
random_forest_model = RandomForestClassifier(random_state=42)
random_forest_model.fit(X_train, y_train)
y_pred = random_forest_model.predict(X_test)
```

```
conf_matrix = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:')
print(conf_matrix)
```

Confusion Matrix:

```
[[77 22]  
 [21 34]]
```

```
plt.figure(figsize=(4, 3))  
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="cubehelix", cbar=False,  
            xticklabels=['No Diabetes', 'Diabetes'], yticklabels=['No Diabetes', 'Diabetes'])  
plt.title('Confusion Matrix-Random Forest')  
plt.xlabel('Predicted')  
plt.ylabel('Actual')  
plt.show()
```



```
_RFaccuracy=accuracy_score(y_test, y_pred)  
precision=precision_score(y_test,y_pred)  
recall=recall_score(y_test,y_pred)  
f1=f1_score(y_test,y_pred)  
conf_matrix=confusion_matrix(y_test,y_pred)  
print(f'RF Accuracy: {accuracy:.4f}')print(f'Precision: {precision:.4f}')print(f'Recall: {recall:.4f}')
```

```
print(f'F1 Score: {f1:.4f}')  
print('Confusion Matrix:')  
print(conf_matrix)
```

```
RF Accuracy: 0.7208  
Precision: 0.6071  
Recall: 0.6182  
F1 Score: 0.6126  
Confusion Matrix:  
[[77 22]  
 [21 34]]
```

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder, StandardScaler  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.svm import SVC  
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score, confusion_matrix
```

```
# Load Dataset  
df = pd.read_csv('Loan Dataset.csv')  
  
# Display first rows  
df.head()
```

	customer_id	customer_age	customer_income	home_ownership	employment_duration	loan_intent	loan_grade	loan_amnt	loan_in
0	1.0	22	59000	RENT	123.0	PERSONAL	C	£35,000.00	
1	2.0	21	9600	OWN	5.0	EDUCATION	A	£1,000.00	
2	3.0	25	9600	MORTGAGE	1.0	MEDICAL	B	£5,500.00	
3	4.0	23	65500	RENT	4.0	MEDICAL	B	£35,000.00	
4	5.0	24	54400	RENT	8.0	MEDICAL	B	£35,000.00	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
# Basic Info
print("Shape:", df.shape)
print("\nColumns:\n", df.columns)

# Missing values
print("\nMissing Values:\n", df.isnull().sum())

# Data types
print("\nData Types:\n", df.dtypes)
```

Shape: (32586, 13)

Columns:

```
Index(['customer_id', 'customer_age', 'customer_income', 'home_ownership',
      'employment_duration', 'loan_intent', 'loan_grade', 'loan_amnt',
      'loan_int_rate', 'term_years', 'historical_default', 'cred_hist_length',
      'Current_loan_status'],
      dtype='object')
```

Missing Values:

```
customer_id      3
customer_age      0
customer_income  0
home_ownership    0
employment_duration  895
```

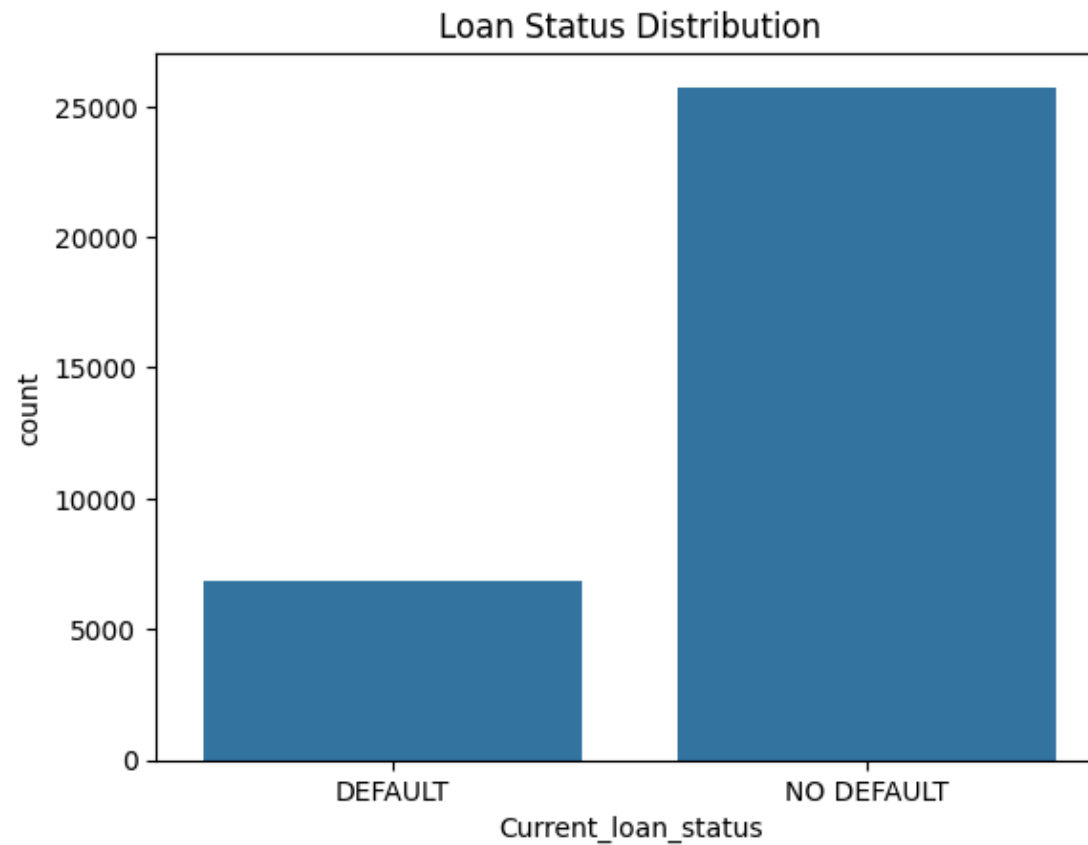


```
loan_intent      0
loan_grade       0
loan_amnt        1
loan_int_rate    3116
term_years       0
historical_default 20737
cred_hist_length 0
Current_loan_status 4
dtype: int64
```

Data Types:

```
customer_id      float64
customer_age      int64
customer_income   object
home_ownership    object
employment_duration float64
loan_intent       object
loan_grade        object
loan_amnt         object
loan_int_rate     float64
term_years        int64
historical_default object
cred_hist_length  int64
Current_loan_status object
dtype: object
```

```
# Distribution of target variable
sns.countplot(x='Current_loan_status', data=df)
plt.title("Loan Status Distribution")
plt.show()
```



```
# Drop missing values
df = df.dropna()

# Encode categorical variables
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

df.head()
```

	customer_id	customer_age	customer_income	home_ownership	employment_duration	loan_intent	loan_grade	loan_amnt	loan_in
0	1.0	22	1379	3	123.0	4	2	346	
2	3.0	25	1914	0	1.0	3	1	394	
3	4.0	23	1527	3	4.0	3	1	346	
4	5.0	24	1263	3	8.0	3	1	346	
5	6.0	21	1939	2	2.0	5	0	229	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
# Define features and target
X = df.drop('Current_loan_status', axis=1)
y = df['Current_loan_status']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Feature Scaling for SVM
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Random Forest Model
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
```

```
# SVM Model
svm = SVC()
svm.fit(X_train_scaled, y_train)
```

```
# Evaluation Function
def evaluate(y_true, y_pred):
    print("Precision:", precision_score(y_true, y_pred, zero_division=0))
    print("Recall:", recall_score(y_true, y_pred, zero_division=0))
    print("F1 Score:", f1_score(y_true, y_pred, zero_division=0))
    print("Accuracy:", accuracy_score(y_true, y_pred))
```

```
print("Random Forest Performance:")
evaluate(y_test, y_pred_rf)
```

```
Random Forest Performance:
Precision: 0.8388429752066116
Recall: 0.9441860465116279
F1 Score: 0.888402625820569
Accuracy: 0.902158273381295
```

```
print("SVM Performance:")
evaluate(y_test, y_pred_svm)
```

```
SVM Performance:
Precision: 0.773678963110668
Recall: 0.9023255813953488
F1 Score: 0.833064949006978
Accuracy: 0.8508393285371703
```

```
# Confusion Matrix for Random Forest
cm = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm, annot=True, fmt='d')
```